Abdiweli Aden yussuf

Exercise A

Prompt 1:Zero-shot

Explain the Software Development Life Cycle (SDLC).

AI Response:

The Software Development Life Cycle (SDLC) is a step-by-step process used to develop software. It helps developers plan, create, test, deploy, and maintain software. The main stages of the SDLC include planning, requirements analysis, design, development, testing, deployment, and maintenance. Following these stages helps ensure that software is developed in an organized and effective way.

Summary (3–5 sentences):

The response explains that SDLC is a process used to develop software. It identifies the main stages, including planning, design, development, testing, deployment, and maintenance. It also explains that SDLC helps developers work in an organized way. The explanation is simple and gives a basic understanding of the topic.

Prompt 2: Role Prompting:

Act as an experienced software engineering teacher. Explain the Software Development Life Cycle (SDLC) to a beginner who is learning software development for the first time. Use simple language and organize your answer with headings and bullet points. Explain each stage of the SDLC and give a simple example for each stage.

AI Response:

Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a structured process that software developers follow to create and maintain software.

1. Planning
Developers identify the problem, goals, resources, time, and cost of the project.
Example: A team decides to create a budget-tracking application.

2. Requirements Analysis
The team identifies what users need the software to do.
Example: Users need to add income, record expenses, and view their spending.

3. Design
Developers plan how the software will look and work.
Example: They design the screens, buttons, database, and overall structure of the budget app.

4. Development
Programmers write the actual code for the software.
Example: Developers use HTML, CSS, JavaScript, or another programming language to build the application.

5. Testing
The software is tested to find and fix errors.
Example: Developers check whether expenses are saved correctly and whether calculations work properly.

6. Deployment
The completed software is released for users.
Example: The budget application is published so users can start using it.

7. Maintenance
Developers continue to fix problems and improve the software after release.
Example: A developer fixes a bug or adds a new feature requested by users.

Summary (3–5 sentences):

This response explains SDLC in more detail and is easier for a beginner to follow. It describes each stage using simple language and provides an example for every stage. The headings and bullet points make the information easier to read. The role of a software engineering teacher also makes the explanation more focused on helping a beginner understand the topic.

Compare

T*he second response was more useful because it was organized specifically for a beginner and explained each SDLC stage with an example. I changed the second prompt by giving the AI a role as an experienced software engineering teacher, specifying that the audience was a beginner, and asking for headings, bullet points, and examples. The role helped the AI explain the topic in a teaching style, while the audience specification made the language simpler. The requested format also made the response easier to read and understand.*

Exercise B: Software Development Models

Prompt:

You are a software engineering advisor. For each of the following four situations, recommend the most appropriate software development model:

1. A project with stable, clearly defined requirements.
2. A startup where requirements change frequently based on user feedback.
3. A large project with significant technical and security risks.
4. A project where testing and verification are extremely important.

For each situation:

- Recommend one development model.
- Explain why it is appropriate.
- Briefly mention one limitation of the recommended model.

Use simple language and organize the answer with headings and bullet points***.***

AI Response:

1. Stable, clearly defined requirements — Waterfall Model

The Waterfall model is appropriate because the requirements are already clear and unlikely to change. The project can move through planning, design, development, testing, and deployment in a structured sequence.

Limitation: Changes can be difficult and expensive once development has started.

2. Frequently changing requirements — Agile Model

Agile is appropriate for a startup because it allows the team to develop the product in small parts and use user feedback to make changes.

Limitation: The project can become difficult to predict in terms of final cost and schedule if requirements keep changing.

3. Large project with technical and security risks — Spiral Model

The Spiral model is suitable because it focuses strongly on identifying and managing risks during development. The team can repeatedly analyze risks, develop a part of the system, and evaluate the results.

Limitation: It can be more expensive and complicated than simpler development models.

4. Testing and verification are extremely important — V-Model

The V-Model is appropriate because testing and verification are planned alongside each development stage. This makes it useful when quality and verification are very important.

Limitation: It is less flexible when requirements change during development.

Reflection :

I learned that the choice of a software development model depends on the characteristics of the project. Stable requirements can work well with a structured model such as Waterfall, while changing requirements are better suited to Agile. Projects with high risks may benefit from the Spiral model, while projects requiring strong testing and verification can use the V-Model. The project requirements, risks, flexibility, and testing needs should therefore be considered before choosing a model.

Exercise C: Requirements Extraction

prompt:

Analyze the following client brief for a local bookstore.

The bookstore needs software to:

- Track books, quantities, and prices.
- Manage supplier payments.
- Alert staff when stock is low.
- Identify best-selling books.
- Allow customers to check availability online.
- Produce clear records for a part-time assistant.
- The owner has a limited budget.

Identify and organize the requirements into these six categories:

1. Functional requirements.
2. Non-functional requirements.
3. Must Have requirements.
4. Should Have requirements.
5. Nice to Have requirements.
6. Questions or ambiguities that need clarification.

Use simple language and explain briefly why each requirement belongs in its category. Consider the limited budget when prioritizing the requirements

AI Response:

1. Functional Requirements

These describe what the system should do:

- Track books, quantities, and prices.
- Manage supplier payments.
- Alert staff when stock is low.
- Identify best-selling books.
- Allow customers to check book availability online.
- Produce clear records for the part-time assistant.

2. Non-Functional Requirements

These describe how the system should work:

- The system should be easy to use.
- The system should provide clear and understandable records.
- The system should protect payment and business information.
- The system should be reliable.
- The system should work well within the bookstore's available budget.

3. Must Have Requirements

- Book and stock tracking.
- Price tracking.
- Supplier payment management.
- Low-stock alerts.
- Clear records.
- A simple and reliable system.

4. Should Have Requirements

- Best-selling book reports.
- Online availability checking.

5. Nice to Have Requirements

- Advanced sales reports.
- Charts and graphs.

- Automatic notifications to customers.
- A mobile application.

6. Questions or Ambiguities

- How many books will the system need to manage?
- What payment methods should be supported?
- Should customers be able to purchase books online or only check availability?
- How should low-stock levels be determined?
- What information should appear in the records?
- What is the exact budget for the system?
- Who will have access to supplier payment information?

Evaluate

Two things the AI did well:

1. It correctly separated the main system functions from quality requirements.
2. It identified useful clarification questions, especially about the budget and online customer access.

One thing the AI missed or categorized incorrectly:

The AI placed the budget under non-functional requirements. The limited budget is mainly a project constraint rather than a direct system quality requirement.

One improvement I would make to the prompt:

I would ask the AI to clearly distinguish between system requirements, project constraints, and optional features so that the categories are more accurate.

Exercise D: Prompt Iteration Challenge

Version 1

Prompt:

Create a project plan for a mobile application that allows university students to book campus study rooms.

AI Response

The project plan could include planning, designing, developing, testing, and launching the study-room booking application. The team would need to identify requirements, design the application, develop the features, test the system, and deploy it for students.

What is missing?

The response is too general. It does not provide specific SDLC phases, time estimates, deliverables, risks, team roles, or a budget.

Version 2

Prompt:

Create a project plan for a mobile application that allows university students to book campus study rooms. Include the main features of the application, the development stages, the team members needed, and an estimated project timeline.

What I changed

I added information about the application's main features, development stages, team members, and timeline.

Why

These details give the AI more context and make the project plan more specific and useful.

Version 3

Prompt:

Act as a software project manager. Create a detailed project plan for a mobile application that allows university students to view available campus study rooms, book rooms, cancel bookings, and receive booking information.

Organize the plan according to the Software Development Life Cycle. Include the main activities, expected deliverables, estimated timeline, team members, and possible project risks. Use simple language and organize the information with headings and tables.

What I changed

I gave the AI a role as a software project manager and added specific features such as viewing rooms, booking, cancellation, and booking information. I also asked for SDLC activities, deliverables, timeline, team members, and risks.

Why

The additional context helps the AI understand exactly what the application should do. The requested structure also makes the project plan easier to understand.

Version 4: Final Prompt

Final Prompt:

Act as an experienced software project manager. Create a detailed project plan for a mobile application that allows university students to view available campus study rooms, book rooms, cancel bookings, and receive booking information.

Organize the project according to all 7 SDLC phases:

1. Planning
2. Requirements Analysis
3. Design
4. Development
5. Testing
6. Deployment
7. Maintenance

For each phase, provide:

- Main activities
- Estimated time
- Key deliverables

Also include:

- Major project risks
- Risk mitigation strategies

- Required team roles and responsibilities
- A rough project budget

Use simple language and organize the response with clear headings and tables. Clearly state any assumptions you make.

AI Response

Project: University Campus Study Room Booking App

SDLC Phase	Main Activities	Estimated Time	Key Deliverables
Planning	Define goals, scope, resources, and schedule	1 week	Project plan
Requirements Analysis	Identify student and university requirements	1–2 weeks	Requirements document
Design	Design the app interface, database, and system structure	2 weeks	UI designs and system design
Development	Build booking, cancellation, availability, and notification features	5–7 weeks	Working application
Testing	Test functionality, security, usability, and performance	2 weeks	Test reports and fixed application
Deployment	Release the application and configure the production system	1 week	Deployed application

Maintenance	Fix problems, monitor the system, and add improvements	Ongoing	Updates and maintenance reports

Key Project Risks and Mitigation

Risk	Mitigation
Double booking of rooms	Use a reliable booking and database validation system
System security problems	Apply authentication, authorization, and security testing
Low student adoption	Test the app with students and collect feedback
Project delays	Use milestones and monitor progress regularly
Budget limitations	Prioritize essential features first

Team Roles

- **Project Manager:** Plans and coordinates the project.
- **Business/Requirements Analyst:** Collects and documents requirements.
- **UI/UX Designer:** Designs the application interface.
- **Mobile Developer:** Develops the mobile application.
- **Backend Developer:** Develops the server and database functions.
- **Tester/QA Engineer:** Tests the application.
- **Security Specialist:** Helps identify and address security risks.

Rough Budget

A rough budget could include:

- Development: $5,000$–**10,000**
- UI/UX design: $1,000$–**2,000**
- Testing and security: $1,000$–**2,000**
- Hosting and deployment: 500–**1,000**
- Maintenance: $1,000$–**3,000 per year**

These figures are estimates and would depend on the team, university resources, technology, and project scope

Final Reflection

Yes, my prompt improved from Version 1 to Version 4. The biggest improvement was adding specific requirements instead of asking for only a general project plan. Adding the 7 SDLC phases, time estimates, deliverables, risks, risk mitigation, team roles, and budget made the final response much more detailed and useful. Giving the AI a role and specifying the format also helped produce a clearer project plan.

Overall Reflection

1. What makes a prompt effective?

An effective prompt is clear, specific, and provides enough context. It should explain what the AI needs to do and, when necessary, specify the desired format and audience.

2. Which prompting technique did you find most useful?

I found role prompting and adding specific requirements most useful because they helped the AI provide a more focused and organized response.

3. Why is context important when working with AI?

Context helps the AI understand the situation and the exact information needed. Without enough context, the response may be too general or may not match the user's needs.

4. Why should we review AI-generated responses instead of accepting them automatically?

AI can make mistakes, misunderstand requirements, or provide inaccurate information. Reviewing the response helps identify errors and ensures that the final information is appropriate.

5. How can AI support software engineers across the SDLC?

AI can help software engineers with requirements analysis, planning, system design, coding, testing, documentation, debugging, and maintenance. It can also help generate ideas and identify possible problems during development.